

32.1 Sorting Summary

Other Desirable Sorting Properties: Stability

A sort is said to be stable if order of equivalent items is preserved.

`sort(studentRecords, BY_NAME);`

Bas	3
Fikriyya	4
Jana	3
Jouni	3
Lara	1
Nikolaj	4
Rosella	3
Sigurd	2

`sort(stude`

Lara
Sigurd
Bas
Jana
Jouni
Rosella
Fikriyya
Nikolaj

Equivalent items don't 'cross over' when being stably sorted.

On the other hand...

sort(studentRecords, BY_NAME);

Bas	3
Fikriyya	4
Jana	3
Jouni	3
Lara	1
Nikolaj	4
Rosella	3
Sigurd	2

sort(stude

Lara
Sigurd
Jouni
Rosella
Bas
Jana
Fikriyya
Nikolaj

Sorting instability can be really annoying! Wanted students listed alphabetically by section.

Arrays.sort() ———

In Java, Arrays.sort(someArray) uses:

- Mergesort (specifically the TimSort variant) if someArray consists of Objects.
- Quicksort if someArray consists of primitives.

```
static void      sort(Object[] a)
                  Sorts the specified array of objects into ascending order, according to the
                  natural ordering of its elements.
```

```
static void      sort(int[ ] a)  
Sorts the specified array into ascending numerical order.
```

► Why?

Sorting

Sorting is a foundational problem.

- Obviously useful for putting things in order.
- But can also be used to solve other tasks, sometimes in non-trivial ways.
 - Sorting improves duplicate findings from a naive N^2 to $N \log N$.
 - Sorting improves 3SUM from a naive N^3 to N^2 .
- There are many ways to sort an array, each with its own interesting tradeoffs and algorithmic features.

Today we'll discuss the fundamental nature of the sorting problem itself: How hard is it to sort?

Sorts Summary

	Memory	# Compares	Notes
Heapsort	$\Theta(1)$	$\Theta(N \log N)$	Bad caching
Insertion	$\Theta(1)$	$\Theta(N^2)$	Best for alm sorted and l
Mergesort	$\Theta(N)$	$\Theta(N \log N)$	Fastest stab
Quicksort LTHS	$\Theta(\log N)$	$\Theta(N \log N)$ expected	Fastest sort



This is due to the cost of tracking recursive calls by the computer, and is also an “expected” amount. The difference between $\log N$ and constant memory is trivial.

You can create a stable Quic
unstable partitioning scheme
partitioning) and using rando
pivots tend to yield better run

32.2 Math Problems Out of Nowhere

A Math Problem

- Is $N! \in \Omega(n^n)$? Prove your answer. [Recall that $\in \Omega$ can be informally be interpreted to mean $\geq$. In other words, does factorial grow at least as quickly as n^n ?

Another Math Problem

- Given: $N! \in \Omega(n^n)$, which we used to prove our answer to the previous problem. Show that $\log(N!) \in \Omega(n \log n)$. [Recall: $\log$ means an unspecified base]

Last Math Problem

- In the previous problem, we showed that $\log(N!) \in \Omega(n \log n)$. Now show that $N \log N \in \Omega(\log(N!))$.

Omega and Theta

- Given $N \log N \in \Omega(\log(N!))$ and $\log(N!) \in \Omega(N \log N)$. Which of the following can we say?

- A. $N \log N \in \Theta(\log(N!))$
- B. $\log(N!) \in \Theta(N \log N)$
- C. Both A and B
- D. Neither

Summary

We've shown that $N \log N \in \Theta(\log(N!))$.

- In other words, these two functions grow at the same rate asymptotically.

As for why we did this, we will see in a little while...

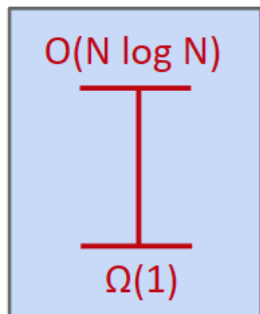
32.3 Theoretical Bounds on Sorting

So far, the fastest sorts we have gone over have had $\Theta(N \log N)$ as the worst case time. This begs the question; can we get a sorting algorithm that would have a better asymptotic analysis?

Let's say that we have "the algorithm ultimate comparison sort" known as TUCS which is the fastest possible comparison sort. We don't know the details of this yet-to-be-discovered algorithm as it has not been discovered yet. Let's additionally say that $R(N)$ is the worst-case runtime of TUCS and give ourselves the challenge of finding the best Θ and O bounds for $R(N)$. (This might seem a little odd since we really don't know the details of it, but we can find $R(N)$).

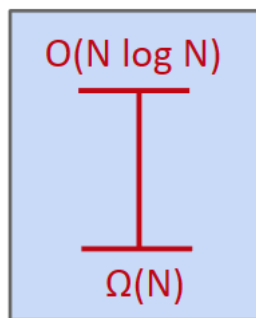
As a starting point, we can go ahead and have the O bound of $R(N)$ be $O(N \log N)$. Why? If this is the fastest sorting algorithm it must be at least just as good as the algorithms we already have! Thus it must at least be as good as Mergesort's worst case as otherwise we would have a contradiction in calling this algorithm TUCS.

What about the starting point for the Ω bound of $R(N)$? Since we don't quite know any details about it we can start by giving it the best possible runtime of $\Omega(1)$. But can we make this tighter?



TUCS Worst
Case Θ Runtime

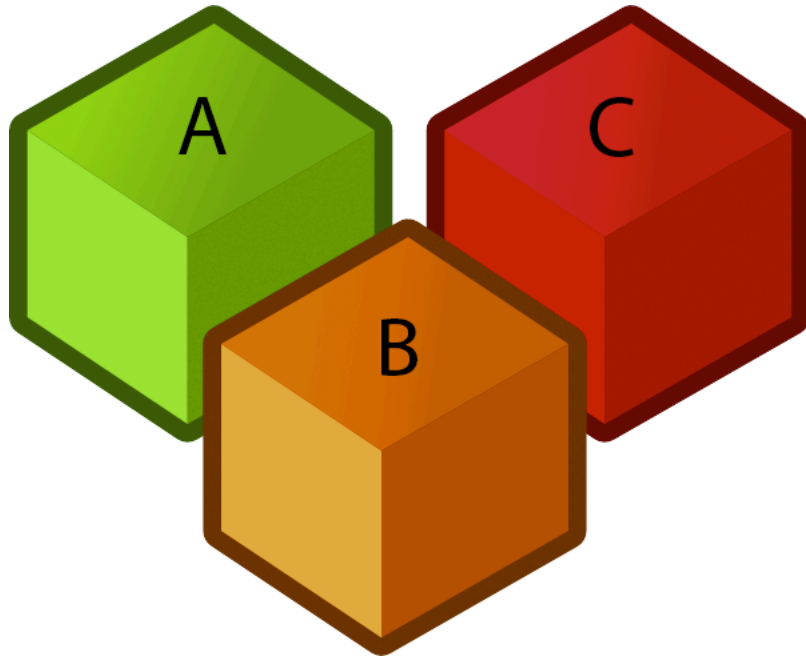
It turns out we can as if the algorithm sorts all of N elements, it must have gone through all of the N elements at some point of sorting it. Thus, we can arrive at a tighter bound of $\Omega(N)$. But can we make it tighter? The answer is yes, but to see the argument for why we have to consider the following game.



TUCS Worst
Case Θ Runtime

A Game of Cat and Mouse (and Dog)

Let's say we place Tom the Cat, Jerry the Mouse, and Spike the Dog in opaque soundproof boxes labeled A, B, and C. We want to figure out which is which using a scale.



Let's say we knew that Box A weighs less than Box B, and that Box B weighs less than Box C. Could we tell which is which?

The answer turns out to be yes!

a < b	b < c		Which is which?
Yes	Yes		a: mouse, b: cat, c: dog (sorted order: abc)

We can find a sorted order by just considering these two inequalities. What if Box A weighs more than Box B, and that Box B weighs more than Box C? Could we still find out which is which?

$a < b$	$b < c$		Which is which?
Yes	Yes		a: mouse, b: cat, c: dog (sorted order: abc)
No	No		c: mouse, b: cat, a: dog (sorted order: cba)

The answer turns out to be yes! So far, we have been able to solve this game with just two inequalities! Let's go ahead and try a third case scenario. Could we know which is which if Box A weighs less than Box B, but Box B weighs more than Box C?

$a < b$	$b < c$		Which is which?
Yes	Yes		a: mouse, b: cat, c: dog (sorted order: abc)
No	No		c: mouse, b: cat, a: dog (sorted order: cba)
Yes	No		

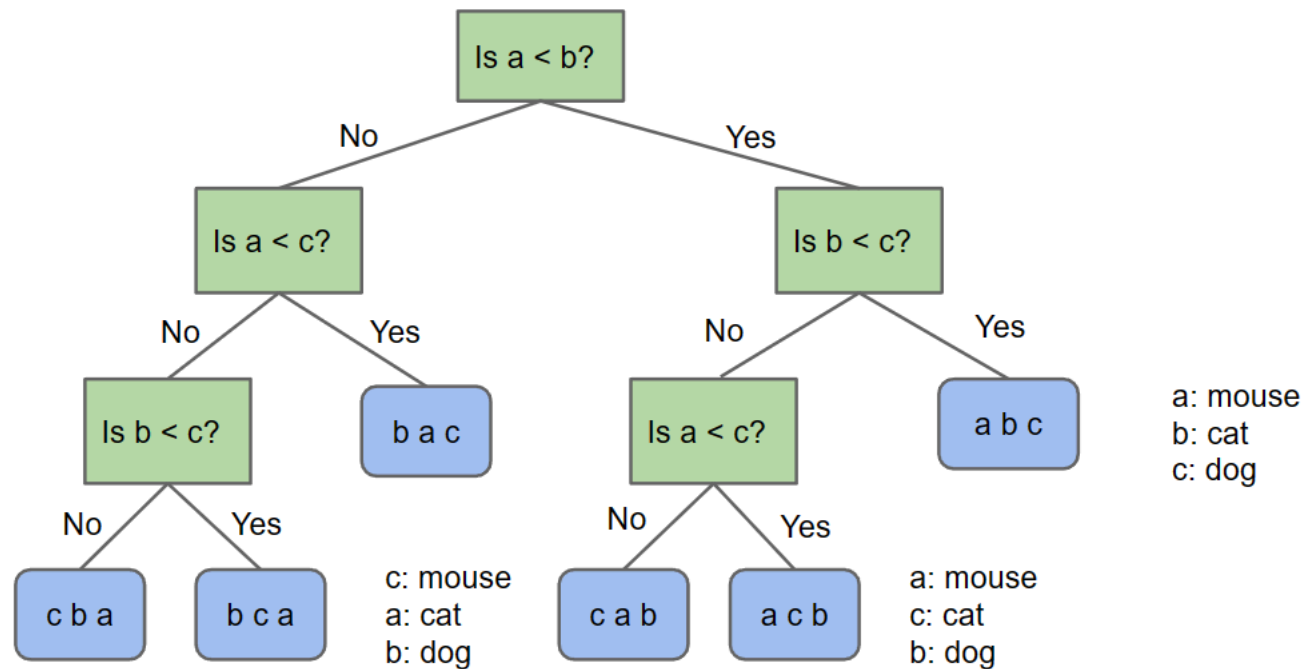
The answer turns out to be no. It turns out to have two possible orderings:

- a: mouse, c: cat, b: dog (sorted order: acb)
- c: mouse, a: cat, b: dog (sorted order: cab)

So while we were on a really great streak of solving the game with only two inequalities, we will need a third to solve all possibilities of the game. If we add the inequality $a < c$ then this problem goes away and becomes solvable.

$a < b$	$b < c$	$a < c?$	Which is which?
Yes	Yes	N/A	a: mouse, b: cat, c: dog (sorted order: abc)
No	No	N/A	c: mouse, b: cat, a: dog (sorted order: cba)
Yes	No	Yes	a: mouse, c: cat, b: dog (sorted order: acb)

Now that we've created this table, we can create a tree to help us solve this sorting game of cat and mouse (and dog).



But how can we make this generalizable for all sorting? We know that each leaf in this tree is a unique possible answer to how boxes should be sorted. So the number of ways the boxes can be sorted should turn out to be the number of leaves in the tree. That then begs the question of how many ways can N boxes be sorted? The answer turns out to be $N!$ ways as there are $N!$ permutations of a given set of elements.

So how many questions do we need to ask in order to know how to sort the boxes? We would need to find the number of levels we must go through to get our answer in the leaf. Since it's a binary tree the minimum amount levels turn out $\lg(N!)$ levels to reach a leaf. (Note that $\lg$ means $(\log \text{ base } 2)$).

So, using this game we have found that we must ask at least $\lg(N!)$ questions about inequalities to find a proper way to sort it. Thus our lower bound for $R(N)$ is $\Omega(\lg(N!))$.

32.4 Summary

Math Problem Out of Nowhere 1. We showed that $N! \in \Omega\left(\frac{N}{2}^{\frac{N}{2}}\right)$.

Math Problem Out of Nowhere 2. We showed that $\log(N!) \in \Omega(N \log N)$, and that $N \log N \in \Omega(\log(N!))$. Therefore $\log(N!) \in \Theta(N \log N)$.

Seeking a Sorting Lower Bound. We've found a number of sorts that complete execution in $\Theta(N \log N)$ time. This raises the obvious question: Could we do better? Let TUCS (which stands for "The Ultimate Comparison Sort") be the best possible algorithm that compares items and puts them in order. We know that TUCS's worst case runtime is $\Theta(N \log N)$ because we already know several algorithm whose worst case runtime is $\Theta(N \log N)$, and TUCS's worst case runtime is $\Omega(N)$ because we have to at least look at every item. Without further discussion, this analysis so far suggest that might be able to do better than $\Theta(N \log N)$ worst case time.

Establishing a Sorting Lower Bound. As a fanciful exercise, we played a game called puppy-cat-dog, in which we have to identify which of three boxes contains a puppy, cat, or dog. Since there are $3! = 6$ permutations, we need at least $\lceil \lg(6) \rceil = 3$ questions to resolve the answer. In other words, if playing a game of 20 questions with 6 possible answers, we have to ask at least 3 questions to be sure we have the right answer. Since sorting through comparisons is one way to solve puppy-cat-dog, then any lower bound on the number of comparisons for puppy-cat-dog also applies to sorting. Given N items, there are $N!$ permutations, meaning we need $\lg(N!)$ questions to win the game of puppy-cat-dog, and by extension, we need at least $\lg(N!)$ to sort N items with yes/no questions. Since $\log(N!) = \Theta(N \log N)$, we can say that the hypothetical best sorting algorithm that uses yes/no questions requires $\Omega(N \log N)$ yes/no questions. Thus, there is no comparison based algorithm that has a worst case that is a better order of growth than $\Theta(N \log N)$ compares.

32.5 Exercises

Factual

1. Which of the following function(s) have the slowest order of growth in terms of Big Theta?
 - ☐ $N \log N$
 - ☐ $\log N!$
 - ☐ N^2
 - ☐ $N! \log N!$
2. To solve puppy, cat, dog for 12 items, what is the theoretical minimum number of comparisons we have to make, based on the argument used in lecture? Please round your answer up to the nearest whole number.
3. Which of the following statements are true?
 - ☐ The optimal sorting algorithm takes at most $\Theta(N \log N)$ time.
 - ☐ It is impossible to find a comparison-based sorting algorithm that takes less than $\Theta(N \log N)$ comparisons.
 - ☐ It is impossible to find a comparison-based sorting algorithm that takes less than $\Theta(N \log N)$ time.
 - ☐ It is impossible to find a sorting algorithm that takes less than $\Theta(N \log N)$ time.

- Problem 1
- Problem 2
- Problem 3

Metacognitive

1. Suppose we add a new method to `Arrays.sort` that takes in an array of strings. What algorithm should `Arrays.sort(String[] x)` use?

- Problem 1

32. Sorting and Algorithmic Bounds